

KeyQuorum Lab

Browser GUI Manual

Bailey Forbes

A guide to KeyQuorum Lab's own browser interface: its header and tabs, and the guided, gated tutorials built into the page. For the `keyquorum` / `keyquorum-device` command-line tool the lab runs underneath, see the separate KeyQuorum manual.

Contents

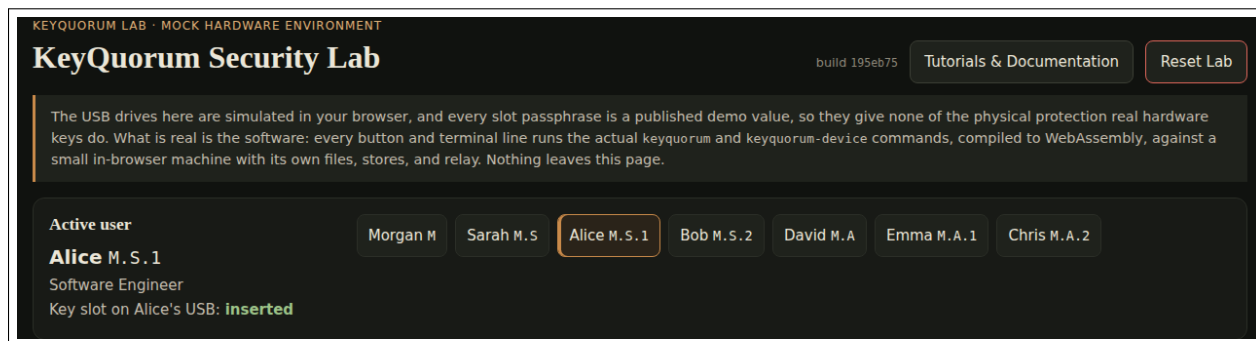
1	Overview	1
1.1	Demo credentials	1
1.2	Identity and drives	2
2	The tabs	2
2.1	Organization key tree	2
2.2	Mock USB devices	3
2.3	File Explorer	4
2.4	Inbox and sent	4
2.5	Security & devices	4
2.6	Activity and access trace	5
2.7	Terminal	5
3	Guided tutorials	5
3.1	How a step works	6
3.2	The modules	7
3.2.1	Identities & drives	7
3.2.2	Files & unlocking	7
3.2.3	Mailbox: sending & receiving	7
4	Where the lab lives	7

1 Overview

KeyQuorum Lab runs the KeyQuorum crate in a browser as a small sandboxed machine, compiled to WebAssembly. It has its own files, a SQLite store per person, mock USB drives, and a mailbox relay answering in the page. Every button and every terminal line runs the real `keyquorum` and `keyquorum-device` commands — there is no separate “demo mode” standing in for them.

Note

This manual documents the lab’s own GUI: its header, tabs, and guided tutorials. It does not re-document `keyquorum` / `keyquorum-device` themselves — that is the separate *KeyQuorum* manual, which the lab runs underneath its interface unchanged. Where a control here is a thin wrapper over a CLI concept (custody, quorum, private bridges, authenticated updates, the mailbox relay), this manual names that concept without re-explaining it.



The lab header: build commit, the Tutorials & Documentation and Reset Lab buttons, the disclaimer, and the Active user bar.

Reset Lab discards everything the session has done — every drive move, sent letter, provisioned key, password file, export, share link, signature, and activity-log entry — and replaces it with a fresh copy of the seeded demo state, instantly. Nothing here is persisted outside the browser tab, so this is the only way back to a known starting point; it is also what **Exit tutorial** and the tutorial picker leave alone; a gated tutorial in progress does not survive a reset.

The mock USB drives are simulated entirely in the browser, and every slot passphrase is a published demo value (Section 1.1), so they give none of the physical protection real hardware keys do. What is real is the software underneath: every action, in every tab, is one of the same commands documented in the KeyQuorum manual.

1.1 Demo credentials

Every credential the lab uses is a published placeholder, not a secret. The lab ships as a public WebAssembly bundle that runs entirely in your browser, so anything baked into it — the seeded people, their USB passphrases, and the seeded files — is visible to anyone who inspects the bundle. That is by design: none of it guards real data, so nothing here needs to stay confidential, and no real file or hardware key is ever at risk of passing through the lab.

USB slot passphrases. Every seeded USB slot unlocks with `lab-demo-<label>`, where `<label>` is that slot’s dotted tree label:

Person	Label	Passphrase
Morgan	M	lab-demo-M
Sarah	M.S	lab-demo-M.S
Alice	M.S.1	lab-demo-M.S.1
Bob	M.S.2	lab-demo-M.S.2
David	M.A	lab-demo-M.A
Emma	M.A.1	lab-demo-M.A.1
Chris	M.A.2	lab-demo-M.A.2

Terminal demo values. Typing a command yourself at the Terminal tab — rather than clicking a GUI action that stages your own value first — is answered with the same published stand-ins a real prompt would use, whenever it asks for a password or PIN:

- Password: `lab-demo-password`
- PIN: `0000`

Note

Seeded quorum-protected files never use a password. They are protected by KeyQuorum's quorum and device-custody policies instead, and are unlocked with the USB identities above.

1.2 Identity and drives

The **Active user** bar shows who you are acting as right now, and which mock USB drive their personal key slot lives on. Clicking another name's chip switches to them, the way sitting down at a different desk would. Most actions need that person's drive *inserted* first — a mock drive behaves like a real one: its slots are only usable while it is connected, and moving a slot onto someone else's drive needs both drives inserted at once, matching the physical requirement of moving a token between two real USB drives.

2 The tabs

At desktop width every tab's panel is visible at once, stacked on one scrolling page; the tab row itself only appears at phone width, where it switches which single panel is shown. What follows is every panel, in the order they appear on the page.

2.1 Organization key tree

The org-wide split tree, drawn as the dotted-label hierarchy it is: `M`, `M.S`, `M.S.1`, and so on. You only ever see your own slice — your lineage, your descendants, siblings, and established-bridge peers. A leaf tagged *required* contributed a share the last time you unlocked a file; *satisfied* means that share was actually presented.

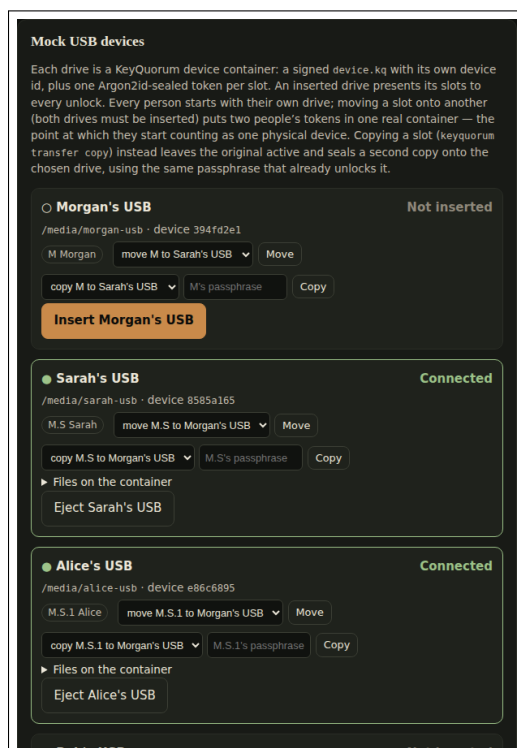
- **Revoke key** bans a leaf's hardware key outright; it drops that leaf's bindings and pairings but does not evict it or refresh survivor shares.
- **Reissue...** replaces a leaf's hardware key onto an already-provisioned replacement token.
- **Bridges** whitelists and establishes cross-branch links, the same two-step allow-then-add process the CLI uses.
- **Tree restructure** walks through proposing a new public generation and collecting the parent's countersignature when one is required.



The organization key tree, showing the dotted-label hierarchy, per-leaf slot status, and the Revoke/Reissue controls.

2.2 Mock USB devices

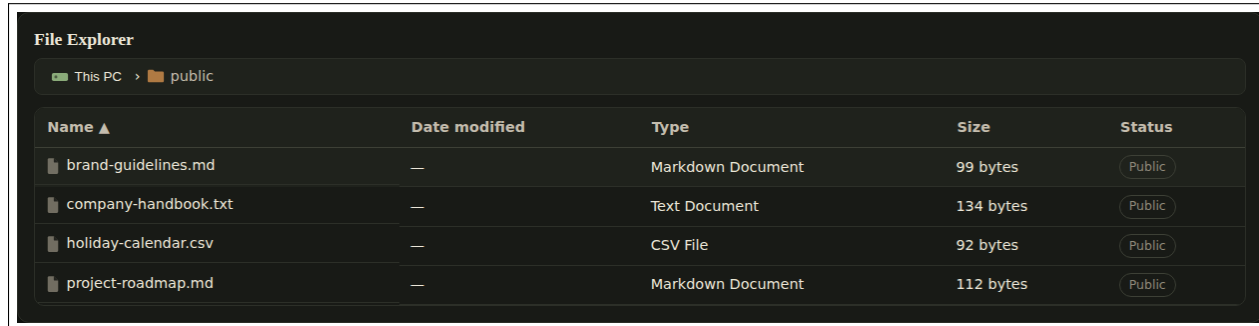
Each card is one device container: a signed device descriptor plus one sealed token per slot. **Insert** / **Eject** plug a drive in or pull it out. **Move** asks for a destination drive (both drives must be inserted) and relocates a slot's keypair onto it — the point where two people's tokens land in one physical container and start counting as a single device toward any multi-device quorum. **Copy** instead leaves the original slot active and seals a second copy elsewhere, using the same passphrase that already unlocks it — a seeded slot's own published demo passphrase (Section 1.1).



Mock USB devices: connected state, the slots each drive carries, and the Move / Copy controls.

2.3 File Explorer

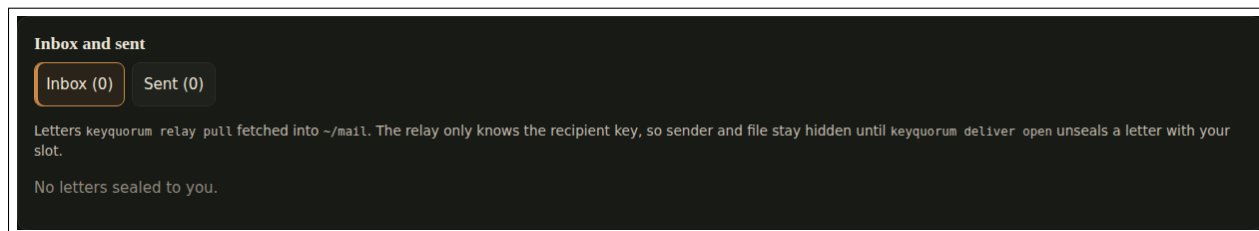
A Windows-Explorer-style view onto the active user's files, grouped into folders. Double-clicking a file (or selecting it and pressing **Open**) unlocks and views it — a quorum file prompts for the shares it needs, a password file for its password. **Send...** seals a copy to another label's key and drops it in their inbox; **Sign** is only offered on public or received plaintext; **Properties** shows the file's protection, size, and timestamps. Right-clicking a row offers the same actions. A file with an expiry stamp shows a status pill that flips to *expired* on its own while the tab is open.



The File Explorer, showing the public folder's contents and their status pills.

2.4 Inbox and sent

Letters are sealed to a recipient's key and routed by the mailbox relay without it ever reading them — sender and contents stay hidden until **Receive** unseals a letter with your own slot. **Reject** answers with a signed refusal instead. The Sent view tracks whether a delivery is still awaiting the recipient's signed acknowledgement, and **Check relay for acknowledgements** pulls any that are waiting.



Inbox and sent, in the empty seeded state.

2.5 Security & devices

Everything that is not the hardware-key quorum tree or a file: locking a note with a password (and optionally a PIN), exporting a file or credential as a portable sealed bundle, minting a time-limited share link, provisioning a fresh slot on an inserted drive and registering it as a new leaf, signing or verifying with a private sign bridge you belong to, and, further down, read-only device logs and relay counters for the in-process mailbox this lab runs against.

Security & devices: the password-lock form, exported bundles, share links, and key/leaf provisioning.

2.6 Activity and access trace

Every action taken leaves a trace here: which checks ran, in order, and whether each one passed. The **Equivalent operation** line under the latest trace is the literal command line that action ran — the same one that could be typed in the Terminal tab. The full log below keeps every action for the session, not just the last one.

2.7 Terminal

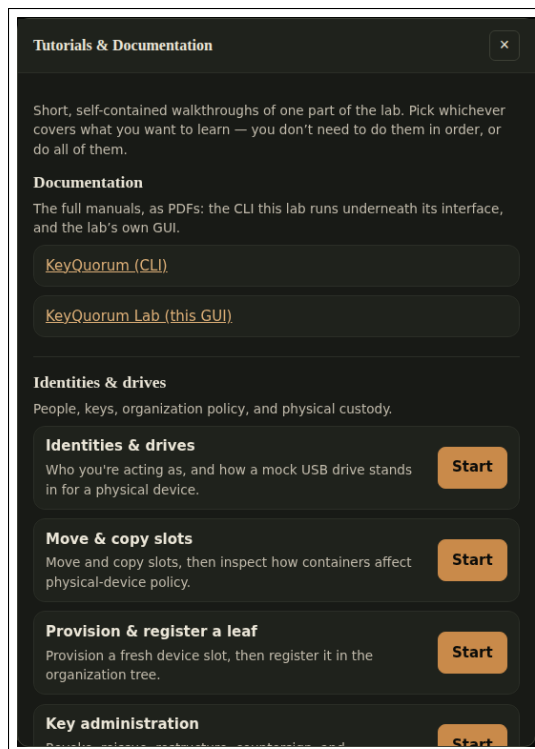
A real shell on the same lab machine. Every button on every other tab runs a command here under the hood; typing it directly works identically, including commands no button exposes yet.

```
keyquorum-device list /media/alice-usb
bridge list 1
usb insert david
su david
```

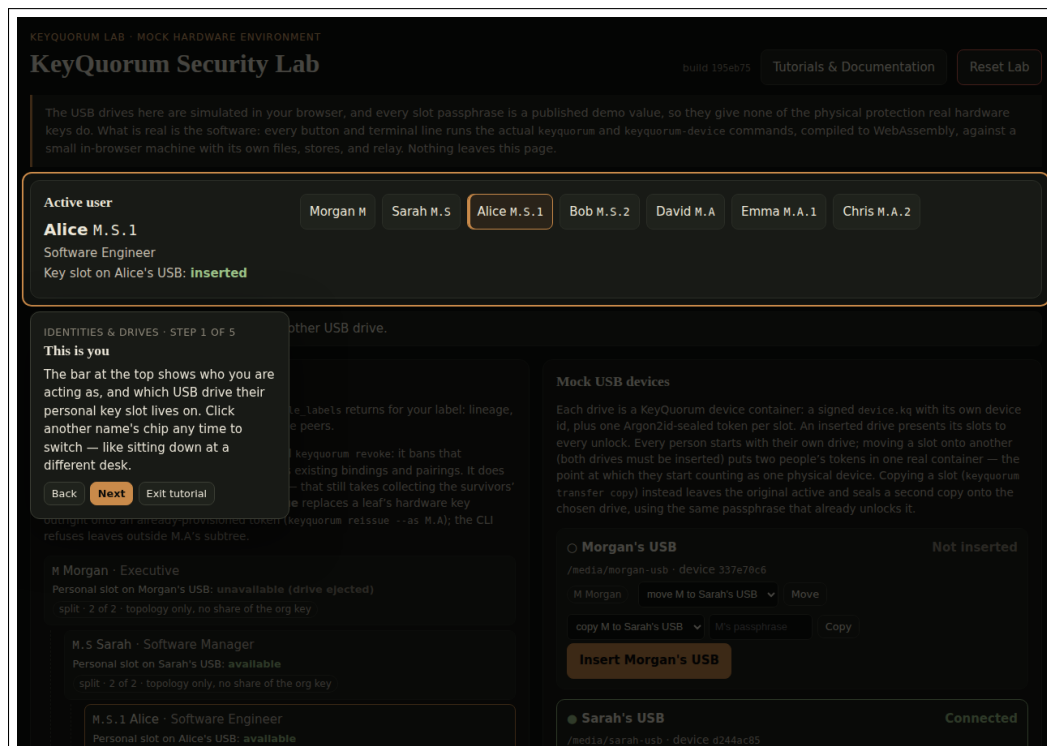
`usb insert` / `usb eject` and `su` are lab-only shell verbs for drive and identity control; every other line is an ordinary `keyquorum` / `keyquorum-device` command. Any passphrase, password, or PIN prompt a typed command raises is answered with its published demo value (Section 1.1), the same way a GUI action's prompt is when you have not staged your own answer first.

3 Guided tutorials

The **Tutorials & Documentation** button in the header opens a picker of short, self-contained modules, plus a **Documentation** section linking the same two PDF manuals this crate ships everywhere else — the CLI manual and this one. Each module covers one part of the lab; picking one is not a commitment to the others, and they can be taken in any order.



The picker: its Documentation section (both PDF manuals) above the Identities & drives category.



A tutorial step in progress: the Active user bar is spotlit, and the tooltip names the step and what to do.

3.1 How a step works

Each step spotlights the part of the page it is about and shows a tooltip with an explanation. A step comes in one of two kinds:

- An **informational** step just explains something; a **Next** button advances it.

- A **gated** step (marked “Waiting for you to try it...”) names a real action to perform — insert a drive, open a file, switch identity, send a letter — and does not advance until that action actually happens in the lab. Completion is checked against the lab’s own state (the current snapshot, or the most recent entry in the Activity log), not against a click on the tooltip itself, so there is no way to fake having done it. Once satisfied, the step marks itself done and advances on its own a moment later. **Skip this step** is always available if you would rather move on without doing it.

A step can also relocate its target as the page changes under it — for example, tracking a folder tile in the File Explorer until it is opened, then tracking the file row that takes its place — so one step can walk you through a short sequence rather than only ever pointing at one fixed element.

3.2 The modules

The picker groups its thirteen modules into three categories; taking one module is never a commitment to the others in its category, or to the category itself.

3.2.1 Identities & drives

- **Identities & drives** — who you’re acting as, bringing a mock USB drive online, reading the organization key tree’s own slice, and switching identity.
- **Move & copy slots** — move and copy slots, then inspect how containers affect physical-device policy.
- **Provision & register a leaf** — provision a fresh device slot, then register it in the organization tree.
- **Key administration** — revoke, reissue, restructure, countersign, and understand parent approval.
- **Bridges & remote devices** — manage cross-branch visibility and distinguish local device actions from relay transfer.

3.2.2 Files & unlocking

- **Files & unlocking** — opening a public file, then trying a quorum-protected one to see what the access trace looks like when a single identity does not meet its threshold.
- **Complete a quorum unlock** — read a file’s requirements, connect cooperating devices, and complete an unlock.
- **Passwords & PINs** — lock a file with the demo password `lab-demo-password`, add a PIN, then unlock it again with the same values (Section 1.1) to see the lab’s second, independent protection scheme end to end.
- **Export bundles & share links** — contrast recipient-sealed export bundles with bearer share links.
- **Signatures, properties & expiry** — sign and verify through a bridge, inspect properties, and observe destructive expiry.

3.2.3 Mailbox: sending & receiving

- **Mailbox: sending & receiving** — sealing a file to another label, switching to that identity, and receiving it from the inbox.
- **Reject & acknowledge** — exercise the rejection path and explicitly refresh a sender’s acknowledgement state.
- **Updates & device letters** — see which higher-level workflows use sealed letters and where their controls actually live.

Finishing a module’s last step opens a short completion notice; from there, or from **Exit tutorial** at any point, the **Tutorials & Documentation** button reopens the picker for another module.

4 Where the lab lives

KeyQuorum Lab is published from `lab/` to GitHub Pages and embedded by `bailey-forbes.com`. A GitHub Actions workflow tests, builds, and publishes it from `main` on every change. The lab is a sandboxed machine, not a second implementation of KeyQuorum: it must never reimplement quorum, custody, approval, visibility, bridge, or delivery rules, or add a gate of its own in front of them — if the CLI would allow something on a real machine, the lab allows it too.